AI_CAMPUS · 온디바이스 AI · Linux & Shell

Day 2

스케줄링 — 누가, 얼마나, 얼마나 늦게

2026.10.06 (화) · 8 교시 · 운영체제 수업 VM (Ubuntu Server 22.04 · 커널 5.15 · vCPU 2)

```
$ ./02_nice_race 10
A (nice 0 ) :  90.3 %   예상  90.3 %
B (nice 10) :   9.7 %   예상   9.7 %
```

오늘의 도착점

스케줄러의 결정을 C 프로그램으로 측정한다

1 교시	2 교시	3 교시	4 교시	5 교시	6 교시	7 교시	8 교시
컨텍스트 스위치	**CFS 실측**	**CPU affinity**	**실시간 정책**	**주기 실행**	**cgroup**	**jitter**	**정리**
01_ctxsw	02 · 03	04_affinity	05_rt_policy	06_sleep_drift	07_busy	jitter.c	정책 고르기

오전 : 얼마나 받는가 (공평 · 위치)

오후 : 얼마나 늦게 받는가 (실시간 · 주기 · 한도 · 지연)

실습 폴더 ~/linux-course/day2/src · sudo 가 필요한 실습 : 05 · jitter -f · cgroup · 두 번째 PuTTY 창 (Duplicate Session) 준비

CPU 를 내려놓는 두 가지 방법

자발적 (voluntary)

스스로 잠들며 내려놓음

sleep · I/O 대기 · 이벤트 대기

1000 sleep 1000 번

비자발적 (nonvoluntary)

할 일이 남았는데 빼앗김

선점 (preemption)

270 busy + stress-ng

```
$ grep ctxt /proc/self/status
voluntary_ctxt_switches:        3
nonvoluntary_ctxt_switches:     0
```

▶ 애니메이션 D2-1 컨텍스트 스위치

vruntime 이 가장 작은 태스크를 고른다

vruntime 증가량 = 실제 실행 시간 × 1024 / weight

nice	weight	둘이 경쟁할 때 nice 0 대비 몫
-5	3121	75.3 %
0	1024	50 %
5	335	24.7 %
10	110	9.7 %
19	15	1.4 %

```
$ ./02_nice_race 10
A (nice 0 ) : 73120810 회   90.3 %   예상 90.3 %
B (nice 10) :  7884416 회    9.7 %   예상  9.7 %

$ nice -n 10 ./03_vruntime
  구간 실행시간(ms) vruntime(ms)   비율
     1      498.2       4637.5   9.31
```

▶ 애니메이션 D2-2 CFS vruntime

어디서 돌게 할 것인가 — 예측성 vs 유연성

방법	예
실행할 때	taskset -c 0 ./prog
실행 중인 PID	taskset -cp 1 <PID>
C 코드	sched_setaffinity(0, sizeof(set), &set)
지금 CPU	sched_getcpu()

자유 — 덜 바쁜 CPU 로 이동 — **100 %**

바쁜 CPU 0 고정 — CPU 1 이 놀아도 대기 — **50 %**

빈 CPU 1 고정 — 혼자 · 예측 가능 — **100 %**

CPU 0 에만 stress-ng 가 돌 때 ./07_busy 의 계산량

▶ **애니메이션** D2-3 CPU affinity

SCHED_FIFO 는 잠들 때까지 비키지 않는다

정책	우선순위	규칙
SCHED_OTHER	nice -20~19	CFS — 공평하게
SCHED_FIFO	1~99	끝나거나 잠들 때까지
SCHED_RR	1~99	FIFO + 같은 순위끼리 시간 조각

```
$ ./05_rt_policy
sched_setscheduler: Operation not permitted
$ sudo ./05_rt_policy 50
변경 후 정책=SCHED_FIFO, 우선순위=50
$ chrt -p 2410
```

RT throttling

950 ms

/ 1000 ms 까지만 RT 실행

무한 루프 RT 가 있어도 50ms 는 셸 · sshd 몫
→ PuTTY 가 멈추지 않는다

▶ **애니메이션** D2-4 실시간 정책

"1ms 쉬기" 가 아니라 "다음 1ms 시각까지"

상대 nanosleep

1303.0 ms 오차 +303 ms

1ms × 1000 번 (목표 1000 ms)

절대 clock_nanosleep

1000.1 ms 오차 +0.1 ms

1ms × 1000 번 (목표 1000 ms)

```
clock_gettime(CLOCK_MONOTONIC, &next);
for (;;) {
    work();
    next.tv_nsec += 1000000;            /* 1 ms */
    if (next.tv_nsec >= 1000000000L) { next.tv_nsec -= 1000000000L; next.tv_sec++; }
    clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &next, NULL);
}
```

▶ 애니메이션 D2-5 주기 실행과 드리프트

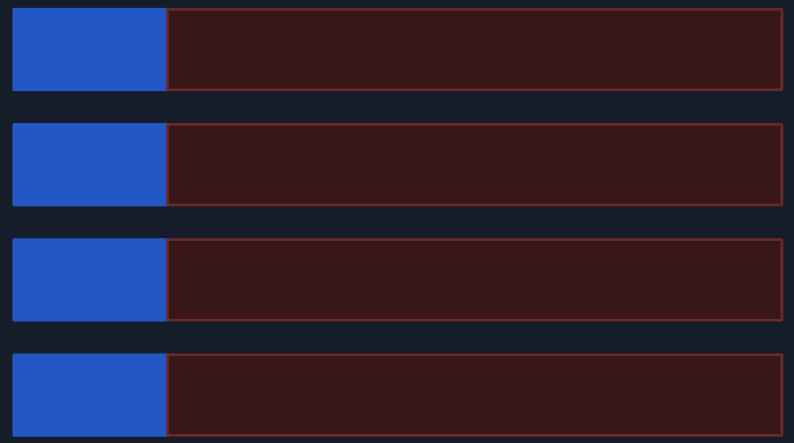
cpu.max — 주기마다 쓸 수 있는 시간

파일	의미	예
cpu.max	주기마다 쓸 시간	20000 100000 = 20 %
cpu.weight	경쟁할 때 상대 몫	기본 100
cpu.stat	사용량 · 제한 횟수	nr_throttled
cgroup.procs	소속 PID	echo PID > …

```
$ sudo systemd-run --scope -p CPUQuota=20% ./07_busy
Running scope as unit: run-r3f1c….scope
  1초:    5323011 회

$ cat /sys/fs/cgroup/lab/cpu.max
30000 100000
```

100ms 주기 · quota 20ms



파랑 = 실행 · 빨강 = throttled

▶ 애니메이션 D2-6 cgroup cpu.max

지연은 평균이 아니라 꼬리를 본다

조합	평균	99%	최대 (예시, us)
OTHER · 부하 없음	95	160	900
OTHER · 부하	2,100	3,900	8,000
FIFO 80 · 부하	45	110	600
FIFO 80 · CPU 1 · 부하	40	95	450

```
$ stress-ng --cpu 2 &
$ ./jitter -o other_load.csv
$ sudo ./jitter -f 80 -c 1 -o fifo_cpu1.csv
```

판단 기준

- 부하를 건 상태에서
- 최대 · 99 % 로
- 여러 번 측정

▶ **애니메이션** D2-7 jitter 측정

임베디드에서 정책을 고르는 기준

1. 주기가 짧고 늦으면 안 되는 일만 RT — 모터, 센서, 오디오
2. RT 태스크에는 반드시 잠드는 지점 (clock_nanosleep)
3. 우선순위는 필요한 만큼만 — 99 는 커널 스레드 몫
4. 무거운 계산 (AI 추론, 로그) 은 OTHER 로, 다른 CPU 에
5. 부하를 건 상태의 최대 지연으로 판단

명령	하는 일
nice / renice	nice 값
taskset	CPU 고정
chrt	RT 정책
systemd-run	cgroup 한도

▶ **애니메이션** D2-8 스케줄러 놀이터

다음 시간 · Day 3 (10/7 수)

메모리

가상 주소 공간 · demand paging · mmap · Copy-on-Write 를 page fault 수로 증명한다

개념 워크시트 80 문항
Linux-day2/worksheet.html

애니메이션 8 종
Linux-day2/animation.html